

GridWise — Solution Delivery Plan

LLM-Assisted Smart Campus Energy Optimization · BUP CSE Fest 2026 Hackathon
(Online Preliminary)

An industry-grade, production-style architecture and execution plan for building, guardrailing, deploying and submitting the GridWise **POST /optimize-energy** service within the 4-hour preliminary window — engineered for correctness-first scoring (LLM interpretation → deterministic guardrails → mathematical optimization → replay validation) and built to generalize cleanly to unseen, paraphrased operator notes.

Plan Dimension	Decision
Target service	FastAPI HTTP API — GET /health, POST /optimize-energy
Core pipeline	LLM Interpreter → Guardrail Validator → LP/MILP Optimizer → Final Replay Validator → API Response
Optimization engine	Mixed-Integer / Linear Program (OR-Tools CP-SAT or PuLP/CBC), 24-hour horizon
LLM strategy	Structured-output extraction with strict JSON schema + function calling, temperature 0, self-consistency guard
Deployment	Docker (multi-stage, non-root, 0.0.0.0 bind) + one-click cloud deploy (Render/Fly.io/Railway) as primary, Docker as fallback
Round window	7:00 PM – 11:00 PM (4 hours) — time-boxed plan in Section 10

Prepared as a reference execution plan — architecture, guardrail design, optimizer formulation, API contracts, testing strategy, deployment path, and a minute-by-minute timeline aligned to the official Problem Statement and Participant Guide & Evaluation Rubric.

Contents

1. Guiding Principles & Scoring Alignment
2. System Architecture — End-to-End Pipeline
3. Technology Stack (Industry-Standard, Futureproof)
4. LLM Interpretation Layer — Design
5. Deterministic Guardrail Validator
6. Optimization Engine — Mathematical Formulation
7. API Contract & Service Implementation
8. Reliability, Performance & Observability
9. Testing Strategy
10. 4-Hour Execution Timeline
11. Deployment & Docker Fallback
12. Documentation, Repository & Submission Checklist
13. Risk Register & Mitigations
14. Stretch Goals (If Time Remains)

1. Guiding Principles & Scoring Alignment

The rubric rewards a **pipeline**, not a single clever component: understand the note → validate the structured directive → apply it to optimization → return a replay-valid schedule → then minimize cost. The plan below is ordered to build points in exactly that priority.

Principle	Why it drives points
Correctness before cost	A cheap-but-invalid schedule scores zero optimization credit. Guardrails and replay validation are built before any cost-tuning.
Untrusted LLM output	Every LLM response is treated as untrusted structured data; nothing reaches the optimizer without passing schema + semantic guardrails.
Determinism where it matters	Directive validation, optimization, and final replay are 100% deterministic code — only note interpretation touches the LLM.
Paraphrase robustness	Prompted extraction (not keyword/regex matching) so hidden reworded notes map to the same directive as public examples.
Fail safe, never fail hard	Malformed LLM output, provider errors, or bad input degrade to a safe no_op / controlled error — never a crash or invented directive.
Reproducibility as a feature	One-command local run, one docker run fallback, and a README a stranger can follow with zero Slack messages to the team.

This plan intentionally mirrors the 100-point rubric weighting: LLM Interpretation (25) and Directive Application (25) receive the deepest engineering investment, Optimization Quality (10) is solved with a real solver rather than heuristics, and Deployment/Docs/Docker (10+10) are treated as first-class deliverables, not afterthoughts.

2. System Architecture — End-to-End Pipeline

A linear, inspectable pipeline with a hard boundary between the language layer and the math layer, matching the canonical flow in the Problem Statement (Energy Data + Notes → LLM Interpreter → Guardrail Validator → Math Optimizer → Final Validator → API Response).

Stage	Responsibility
1. Ingress / Schema Guard	FastAPI + Pydantic v2 models validate the request shape (24 unique hours, 1–3 notes, battery object) before any processing. Malformed JSON → 400 immediately.
2. LLM Interpreter	One structured-output call per request (batched across all notes) returns a candidate directive_interpretation array. Strict JSON schema / tool-calling forces shape; temperature 0.
3. Guardrail Validator	Deterministic Python: enforces allowed directive_type enum, note_index coverage/order, hour uniqueness/range/ascending order, factor $\in [0,1]$, non-negative finite numerics, applies-semantics. Rejects/repairs, never trusts blindly.
4. Directive Compiler	Converts validated directives into optimizer-ready constraint objects (effective solar override, reserve floor, charge/discharge blocks, grid caps) per Section 5.3 of the spec.
5. Math Optimizer	24-hour LP/MILP (OR-Tools CP-SAT or PuLP+CBC) minimizes grid cost subject to energy balance, battery bounds/rates, and every compiled directive constraint.

Stage	Responsibility
6. Final Replay Validator	Independently re-simulates the returned hourly_plan hour-by-hour (energy balance, battery transitions, directive satisfaction, end-of-day neutrality) before the response leaves the service — a self-check mirroring the judge harness.
7. Response Assembler	Recomputes total_grid_kwh, total_cost_bdt, peak_grid_kwh from hourly_plan (never trusts solver-reported totals) and generates a short plan_summary.

Data flow diagram (textual):

Client JSON → [Pydantic schema guard] → [LLM Interpreter: notes → candidate directives] → [Guardrail Validator: enum / hours / applies / numeric checks] → [Directive Compiler: constraint objects] → [Optimizer: CP-SAT / CBC solve] → [Replay Validator: independent re-check] → [Response Assembler] → JSON

*Key architectural decision: the LLM is invoked **once per request** with all operator notes passed together in a single structured-extraction call (rather than N separate calls). This halves latency and token cost, keeps p95 well inside the 30-second timeout, and lets the model use cross-note context (useful for distractor detection).*

3. Technology Stack (Industry-Standard, Futureproof)

Layer	Choice & rationale
API framework	FastAPI (async, Pydantic v2 typed models, auto OpenAPI docs, uvicorn/gunicorn ASGI)
Language	Python 3.12
LLM provider	Claude (Anthropic Messages API) or GPT-4o-class model, called with strict JSON-schema / tool-use so output shape is enforced by the API itself, not regex-parsed from free text
Structured output enforcement	Provider-native tool-calling / JSON mode as the first guard; Pydantic model validation as the second, independent guard
Optimization solver	Google OR-Tools CP-SAT (preferred: exact, fast, handles MILP-style rate/window constraints natively) with PuLP + CBC as a lighter fallback
Guardrail layer	Pure-Python deterministic validators — zero ML, unit-testable in isolation
Server	uvicorn (dev) behind gunicorn worker manager (prod) — async I/O so LLM network calls don't block health checks
Containerization	Multi-stage Dockerfile, slim base image, non-root user, binds 0.0.0.0:\$PORT, no baked-in secrets (env-var injected)
Config & secrets	12-factor: all keys/model names via environment variables, .env excluded via .gitignore, documented (names only) in README
Testing	pytest for guardrails/optimizer unit tests; httpx/TestClient for API contract tests; the 10 public sample cases wired as regression tests
Observability	Structured JSON logging (request id, stage timings, directive counts) with secrets redacted; no raw prompts/stack traces in responses
CI (optional, time permitting)	GitHub Actions: lint (ruff), type-check (mypy), pytest, and a docker build smoke test on every push

Why CP-SAT over a hand-rolled greedy heuristic: the directive set (reserve floors, no-charge/no-discharge windows, grid caps, solar factors) turns this into a constrained scheduling problem with hard feasibility requirements. A real solver guarantees a globally optimal, provably feasible 24-hour plan in milliseconds — removing an entire class of "technically-cheap-but-invalid" failures the rubric explicitly zeroes out.

4. LLM Interpretation Layer — Design

Goal: convert 1–3 free-text operator notes into exactly one machine-checkable directive_interpretation entry per note, in note_index order, robust to paraphrasing — while never inventing values the note didn't state.

4.1 Prompting strategy

- **System prompt** encodes the full closed directive taxonomy (solar_reduction, minimum_battery_reserve, no_charge_window, no_discharge_window, max_grid_window, no_op), the exact structured_adjustment shape per type, and the whole-hour, start-inclusive/end-exclusive time convention ("1 PM–3 PM → [13,14]").
- **Few-shot exemplars** drawn from the public sample pack style (not copied verbatim) demonstrate a solar-reduction note, a hard-window note, a reserve note, and a distractor note resolving to no_op — teaching the shape, not the wording.
- **Structured output / tool-calling** is used instead of "return JSON" instructions in plain text, so the provider itself enforces valid JSON with the right field types before it ever reaches Python.
- **Temperature 0** and a fixed system prompt for maximal run-to-run consistency across repeated hidden requests.
- The model is explicitly instructed: *"If a note does not match one of the six directive types, or is ambiguous/unsupported, return no_op — never invent a new directive or estimate missing numbers."*

4.2 Paraphrase robustness

Because hidden notes reword the same rule (percentages vs. fractions, "1–3 PM" vs. "13:00 to 15:00" vs. "the afternoon maintenance window"), the prompt teaches the *semantic pattern* for each directive type rather than trigger phrases, and includes explicit unit-conversion guidance (percentage-reduction → factor, clock time → 24-hour integer hours, inclusive-start/exclusive-end).

4.3 Self-consistency safety net

For any note the guardrail layer flags as malformed (e.g. missing required field, out-of-range factor), the service issues one bounded retry with a corrective follow-up instruction before falling back to a safe no_op for that single note — isolating a single bad extraction from failing the entire request.

5. Deterministic Guardrail Validator

Pure, dependency-free Python functions — independently unit-tested — that sit between the LLM and the optimizer. No LLM output reaches the solver unless every check below passes.

Check	Rule enforced
Type check	directive_type ∈ {solar_reduction, minimum_battery_reserve, no_charge_window, no_discharge_window, max_grid_window, no_op}
Coverage	Exactly one entry per note_index, 0..N-1, no missing/duplicate mappings, returned in order
Applies semantics	no_op ⇒ applies=false, structured_adjustment=null; every other type ⇒ applies=true with the exact required shape

Check	Rule enforced
Hours	Unique integers 0–23, strictly ascending, non-empty
Solar factor	$0 \leq \text{factor} \leq 1$
Reserve / grid cap	Finite, non-negative; reserve $\leq$ battery capacity
No invention	Directive may not alter base demand, tariff, or battery parameters outside its own defined effect
Malformed output	Caught, logged (redacted), single field repaired via bounded retry, or the note is safely demoted to no_op — service never 500s on bad model output

6. Optimization Engine — Mathematical Formulation

24 hourly decision variables per energy flow, solved as a linear program (continuous kWh values) with the validated directives compiled into per-hour bounds — exactly matching Section 09 of the Problem Statement.

Decision variables (per hour $h = 0..23$):

$\text{grid_kwh}[h] \geq 0$ · $\text{solar_used_kwh}[h] \geq 0$ · $\text{charge_kwh}[h] \geq 0$ · $\text{discharge_kwh}[h] \geq 0$ ·
 $\text{battery_energy_after}[h]$ · $\text{is_charging}[h] \in \{0,1\}$ (mutual exclusion)

Objective:

minimize $\sum \text{grid_kwh}[h] \times \text{tariff_bdt_per_kwh}[h]$ for $h = 0..23$

Core constraints:

Constraint	Formulation
Energy balance	$\text{grid_kwh}[h] + \text{solar_used_kwh}[h] + \text{discharge_kwh}[h] = \text{demand_kwh}[h] + \text{charge_kwh}[h]$
Solar cap	$0 \leq \text{solar_used_kwh}[h] \leq \text{effective_solar}[h]$ (effective_solar = base_solar × factor if solar_reduction active, else base_solar)
Battery transition	$\text{battery_energy_after}[h] = \text{battery_energy_after}[h-1] + \text{charge_kwh}[h] - \text{discharge_kwh}[h]$
Battery bounds	$\text{active_min_reserve}[h] \leq \text{battery_energy_after}[h] \leq \text{capacity_kwh}$ (active_min_reserve = max(base minimum_energy_kwh, directive minimum, if reserve directive active in hour h))
Rate limits	$\text{charge_kwh}[h] \leq \text{max_charge_kwh_per_hour}$; $\text{discharge_kwh}[h] \leq \text{max_discharge_kwh_per_hour}$
No-charge / no-discharge windows	$\text{charge_kwh}[h] = 0$ for h in directive hours; $\text{discharge_kwh}[h] = 0$ for h in directive hours
Max grid window	$\text{grid_kwh}[h] \leq \text{max_grid_kwh}$ for h in directive hours
End-of-day neutrality	$\text{battery_energy_after}[23] = \text{initial_energy_kwh}$
Mutual exclusion	$\text{is_charging}[h]$ gates charge_kwh/discharge_kwh so the battery is never simultaneously charging and discharging in one hour

Infeasibility handling: organizer-valid scenarios are guaranteed feasible per the spec, so a solver-reported infeasible status indicates an upstream guardrail miss. In that (defensive) case the service falls back to re-validating directives

once, then serves the best feasible relaxation with a controlled error rather than crashing — logged for post-mortem, never surfaced as a stack trace.

7. API Contract & Service Implementation

Both endpoints are implemented exactly to the canonical Problem Statement schema (Sections 06–10).

Endpoint / Case	Behavior
GET /health	200, {"status": "ok"} — ready within 60s of container start; no external dependency (LLM/solver) check gates it, so it is never a false negative
POST /optimize-energy	Accepts the scenario JSON (scenario_id, operator_notes[1-3], hours[24], battery); returns scenario_id, directive_interpretation, hourly_plan, total_grid_kwh, total_cost_bdt, peak_grid_kwh, plan_summary
400	Malformed JSON or structurally invalid request (wrong hour count, missing fields, wrong types) — caught at the Pydantic layer before any LLM call
500	Controlled internal error (LLM provider fully unavailable after retries, solver exception) — generic safe message only, no secrets or stack traces
Timeout budget	Hard 25s internal deadline (5s headroom under the judge's 30s limit): schema guard ~5ms, LLM call ~1-4s, guardrails ~1ms, solve ~10-100ms, replay ~5ms

*Implementation note: totals (total_grid_kwh, total_cost_bdt, peak_grid_kwh) are always **derived by summation from hourly_plan** in the response assembler — never taken directly from solver internals — so they can never drift from the plan the judge independently replays.*

8. Reliability, Performance & Observability

Concern	Approach
Latency target	p95 ≤ 5s (full 3/3 rubric credit) via async LLM call + millisecond-scale CP-SAT solve
Retry policy	One bounded retry on transient LLM/provider errors with exponential backoff (≤2s), then safe controlled failure — never an unbounded loop
Idempotency	Stateless service; identical input at temperature 0 produces stable output across repeated hidden requests
Concurrency	Async FastAPI + multiple gunicorn/uvicorn workers so one slow LLM call cannot block /health or parallel /optimize-energy calls
Secret hygiene	API keys only via environment variables; structured logs redact note text and model responses that could carry provider error detail; no .env committed
Graceful degradation	Circuit-breaker style short-circuit to safe no_op interpretation if the LLM provider is down for the whole request budget, so the service still returns a valid (conservative) schedule rather than failing outright

9. Testing Strategy

Layer	What it proves
Unit — guardrails	pytest cases for every rejection path (bad enum, out-of-order hours, factor>1, duplicate note_index, negative reserve)
Unit — optimizer	Hand-built tiny scenarios with known-optimal cost to verify the LP formulation before wiring to the API
Contract — API	httpx TestClient asserts schema, status codes, and field presence against the 10 public sample cases
Regression — public pack	All 10 provided cases run automatically; interpretation semantics and replayed plan validity are asserted (not byte-identical schedules, per the equivalence note)
Paraphrase fuzzing	Hand-written reworded variants of each public note (different units, synonyms, time formats) to pressure-test LLM + guardrail robustness before hidden judging
Adversarial input	Malformed JSON, out-of-range hours, empty operator_notes edge cases, and simulated LLM-provider failure to confirm controlled 400/500 behavior, never a crash
External reachability	Post-deploy curl of /health and /optimize-energy from an outside network (not localhost) before submission, per the Participant Guide
Clean-machine reproduction	README followed verbatim on a fresh clone/container to confirm zero undocumented steps

10. 4-Hour Execution Timeline

Scoped to the 7:00 PM – 11:00 PM window, front-loaded on the two 25-point categories so a defensible, scoring submission exists well before the deadline even if later polish is cut.

Time	Block	Deliverable
7:00 – 7:20	Repo bootstrap	New private GitHub repo, FastAPI skeleton, Pydantic request/response models transcribed directly from the Problem Statement, .gitignore for secrets
7:20 – 7:50	Guardrail validators	Write and unit-test every deterministic check in Section 5 in isolation, before any LLM code exists
7:50 – 8:30	LLM interpreter	System prompt + structured-output schema + few-shot exemplars; wire to guardrails; run against all 10 public notes
8:30 – 9:15	Optimizer	Implement the CP-SAT/PuLP formulation from Section 6; validate against 2-3 public cases' expected_output cost/validity
9:15 – 9:35	Replay validator + response assembler	Independent re-simulation of hourly_plan; totals always derived from the plan, not the solver
9:35 – 9:55	Full pipeline integration	Wire all 10 public sample cases end-to-end through the live HTTP API; fix schema/edge mismatches
9:55 – 10:15	Dockerize & deploy	Multi-stage Dockerfile, push image, deploy to hosting platform, verify external reachability of both endpoints
10:15 – 10:35	README & Docker fallback verification	Write the copy-paste local quickstart; do a clean-environment dry run of docker pull/run + /health + one sample request
10:35 – 10:50	Adversarial + latency pass	Malformed input tests, timeout budget check, p95 sanity check under repeated calls
10:50 – 11:00	Submission & buffer	Make repo public, submit endpoint/repo/Docker links, final checklist pass against Section 05 of the guide

Note: the 3-minute solution video is intentionally scoped outside this critical path since it carries zero base points and only matters for tie-break — a short screen-recorded walkthrough can be captured any time before the deadline once the pipeline is stable.

11. Deployment & Docker Fallback

Aspect	Plan
Primary hosting	Any reachable platform with no login/VPN gate (e.g. Render, Fly.io, Railway, or a cloud VM) — judged on behavior, not provider
Dockerfile	Multi-stage build: slim Python base, non-root user, dependencies layer cached separately from app code, EXPOSE documented port, binds 0.0.0.0
Registry	Pushed to Docker Hub or GHCR with an exact immutable tag (not :latest) so the fallback stays pullable and reproducible through judging
No baked-in secrets	LLM API key and model name injected at container run-time via -e / --env-file; documented by name only in README

Aspect	Plan
Verified run command	One exact, tested `docker run` command in the README that exposes the right port and reaches /health
Reachability check	Both endpoints curl-tested from an external network (mobile hotspot / online tool) before submission — never trust localhost-only testing

12. Documentation, Repository & Submission Checklist

README.md contents (self-contained, no team assistance required):

- Problem summary and architecture diagram (the pipeline in Section 2)
- Model/provider or local model identifier and the LLM's exact role in the pipeline
- Required environment-variable *names* (never values/secrets)
- Optimizer/solver used and why
- Exact local run command(s) and exact docker pull/run command(s)
- Health check curl example and one public-sample /optimize-energy curl example with expected shape
- Dependency list and known limitations
- Credits for any external libraries, frameworks, or AI coding assistance used

Final submission checklist:

- Repository created after question reveal, private during the event, made public after the deadline
- Live endpoint reachable externally for GET /health and POST /optimize-energy through the full judging window
- Docker fallback image pullable with a working documented run command, no baked-in secrets
- README passes a clean-machine dry run with zero undocumented steps
- 3-minute video recorded, accessible, and under the time limit (tie-break only, not base score)
- No secret values committed anywhere in the repo, logs, or README

13. Risk Register & Mitigations

Risk	Mitigation
LLM provider outage/rate limit mid-judging	Circuit-breaker fallback to safe no_op interpretation + optional secondary/local model as backup, documented in README
Hidden note paraphrase not recognized	Semantic (not keyword) prompting + paraphrase-fuzzing test suite in Section 9 before submission
Solver infeasibility on an edge case	Spec guarantees feasible organizer scenarios; defensive re-validation + logged controlled error as last resort, never a crash
Latency spike breaching 30s judge timeout	25s internal deadline with headroom, async I/O, millisecond-scale solve, single bounded retry only
Docker image fails to pull/start for judges	Exact tag/digest pin (not :latest) + clean-environment dry run performed before submission
Secret leakage in logs/response	Centralized redaction in the logging layer; generic 500 error bodies; automated secret-scan before making repo public

Risk	Mitigation
Totals drift from hourly_plan	Response assembler always derives totals from the plan by summation — structurally impossible to drift

14. Stretch Goals (If Time Remains)

- GitHub Actions CI (lint + type-check + pytest + docker build) for an extra reproducibility/quality signal
- In-memory LRU cache keyed on (notes hash) to shave latency on repeated/near-duplicate hidden requests
- Golden-file regression snapshots of all 10 public cases wired into CI
- Structured tracing (per-stage timing breakdown) surfaced in logs for live latency debugging during judging
- A minimal OpenAPI-driven Swagger page (FastAPI ships this by default) as a bonus reproducibility aid for organizers

This plan is a build blueprint, not a claim of a finished system — every formulation, prompt, and timing figure should be validated against the live Problem Statement, Participant Guide, and Public Sample Cases during implementation.